Praktikum JavaScript & TypeScript

Modul 5 - Object-Oriented Programming

1. Tujuan Praktikum

Setelah mengikuti praktikum ini, peserta mampu:

- Menjelaskan kapan memakai object literal dan kapan memakai `class`
- Membuat class dengan `constructor`, field, dan method, lalu membuat object darinya dengan `new`
- Menjelaskan isi `this` dan mengapa `this` bisa hilang saat method dijadikan callback
- Melindungi data dengan field privat `#` dan membukanya lewat getter
- Menurunkan class dengan `extends`, memanggil induknya dengan `super`, dan menimpa method
- Menyusun class yang menyimpan banyak object class lain, lalu mencarinya lewat id
- Membaca UML class diagram dan pesan error yang sering muncul

2. Dasar Teori

2.1. Dari Object Literal ke Class

Di Modul 4, satu data disimpan sebagai object literal:

```
const akun = {
  pemilik: "Ani",
  saldo: 0,
  setor(n) {              // method: fungsi yang menempel di object
    this.saldo += n;
    return this.saldo;
  },
};
console.log(akun.setor(5));   // 5
akun.saldo = -999;            // siapa pun bisa mengubah saldo langsung
console.log(akun.saldo);      // -999
```

Listing 1: satu akun sebagai object literal

Object literal punya dua kelemahan jika datanya banyak dan aturannya penting:

1. **Bentuknya ditulis ulang untuk setiap akun.** Seratus akun berarti seratus kali menulis `setor` yang sama.
2. **Tidak ada yang menjaga aturan.** Saldo bisa diubah menjadi negatif tanpa melewati pengecekan apa pun.

`class` menyelesaikan keduanya. Class adalah **cetakan**: bentuk data dan perilakunya ditulis sekali, lalu dipakai untuk membuat banyak object. Setiap object yang dibuat dari class disebut *instance*.

Analogi database:

OOP	Database	Contoh
class	`CREATE TABLE`: definisi kolom	`class Rekening`
instance (object)	satu baris	`new Rekening("1001", "Ani")`
field	kolom	`nomor`, `pemilik`
constructor	`INSERT` beserta nilai default	`constructor(nomor, pemilik, saldoAwal = 0)`
method	*stored procedure* untuk baris itu	`setor(nominal)`

field privat `#`	kolom yang hanya bisa diubah lewat procedure	`#saldo`

Kapan memakai yang mana:

Object literal	Class
Data satu kali pakai, misalnya konfigurasi atau hasil `JSON.parse`	Banyak object dengan bentuk dan perilaku sama
Hanya menyimpan data, tanpa aturan	Ada aturan yang harus dijaga, misalnya saldo tidak boleh negatif
Diolah dengan method array (Modul 3 dan 4)	Perilaku menempel pada datanya sendiri

2.2. class, constructor, dan new

```
class Lampu {
  constructor(ruang) {
    this.ruang = ruang;     // field: dibuat untuk setiap instance
    this.nyala = false;
  }

  saklar() {                // method: ditulis sekali, dipakai semua instance
    this.nyala = !this.nyala;
    return `${this.ruang}: ${this.nyala ? "nyala" : "mati"}`;
  }
}

const dapur = new Lampu("Dapur");
const kamar = new Lampu("Kamar");
console.log(dapur.saklar());   // "Dapur: nyala"
console.log(dapur.saklar());   // "Dapur: mati"
console.log(kamar.saklar());   // "Kamar: nyala"  (kamar punya nyala sendiri)
```

Listing 2: class pertama

Bagian-bagiannya:

Bagian	Arti
`class Lampu { ... }`	Membuat cetakan bernama `Lampu`. Nama class diawali huruf kapital.
`constructor(ruang)`	Fungsi khusus yang dijalankan sekali setiap kali `new Lampu(...)` dipanggil. Parameternya diisi dari argumen `new`.
`this.ruang = ruang`	Membuat field `ruang` pada object yang **sedang dibuat**, diisi parameter `ruang`.
`saklar() { ... }`	Method. Tidak memakai kata `function`, tidak dipisah koma.
`new Lampu("Dapur")`	Membuat object kosong, menjalankan `constructor` dengan `this` = object itu, lalu mengembalikan object tersebut.

Jalannya `new Lampu("Dapur")`:

1. JavaScript membuat object kosong `{}`.
2. `constructor` dijalankan dengan `this` menunjuk object itu dan `ruang = "Dapur"`.
3. `this.ruang = "Dapur"` dan `this.nyala = false` mengisi object menjadi `{ ruang: "Dapur", nyala: false }`.
4. Object itu dikembalikan dan disimpan di `dapur`.

⚠ Perhatian

Class wajib dipanggil dengan `new`. `Lampu("Dapur")` melempar `TypeError: Class constructor Lampu cannot be invoked without 'new'`.

2.3. this: Object di Sebelah Kiri Titik

Di dalam method, `this` berisi **object yang memanggil method itu**, yaitu object di sebelah kiri titik saat pemanggilan. Pada `dapur.saklar()`, `this` adalah `dapur`. Pada `kamar.saklar()`, `this` adalah `kamar`. Method-nya satu, datanya berbeda: `dapur.saklar === kamar.saklar` bernilai `true`.

2.3.1. this hilang saat method dijadikan callback

Ingat Modul 3: callback dipanggil **oleh method array**, bukan olehmu. Jika method diberikan begitu saja sebagai callback, method array memanggilnya **tanpa object di kiri titik**. Akibatnya `this` bernilai `undefined`.

```
const lepas = dapur.saklar;   // hanya mengambil fungsinya, object-nya tertinggal
lepas();                      // TypeError: Cannot read properties of undefined (reading 'nyala')

["a"].map(dapur.saklar);      // TypeError yang sama: map memanggil saklar tanpa dapur
```

Listing 3: this hilang

Cara memanggil	this	Hasil
`dapur.saklar()`	`dapur`	berjalan
`const f = dapur.saklar; f()`	`undefined`	`TypeError`
`arr.map(dapur.saklar)`	`undefined`	`TypeError`
`arr.map(() => dapur.saklar())`	`dapur`	berjalan

Solusinya: bungkus dengan arrow function. Di dalam arrow function, kamu sendiri yang memanggil `dapur.saklar()`, jadi object di kiri titik tetap ada.

```
console.log(["a", "b"].map(() => dapur.saklar()));   // ["Dapur: nyala", "Dapur: mati"]
console.log([5000, 2000].map((n) => akun.setor(n)));  // argumen diteruskan sendiri
```

Listing 4: this tetap ada

Alternatif lain adalah `dapur.saklar.bind(dapur)`, yang membuat fungsi baru dengan `this` dikunci ke `dapur`. Arrow function lebih sering dipakai karena lebih mudah dibaca.

2.4. Getter dan Field Privat

2.4.1. Field privat `#`

Untuk apa: melindungi data supaya hanya bisa diubah lewat method yang memeriksa aturannya. Seperti kolom saldo di database yang hanya boleh diubah lewat procedure `setor` atau `tarik`, bukan `UPDATE` langsung.

Cara kerja: field yang namanya diawali `#` hanya bisa diakses dari **dalam** class yang mendeklarasikannya. Field ini harus dideklarasikan di badan class sebelum dipakai.

```
class Termometer {
  #celsius;                       // deklarasi field privat

  constructor(c) {
    this.#celsius = c;
  }

  get celsius() {                 // getter: dibaca seperti field
    return this.#celsius;
  }

  get fahrenheit() {              // getter boleh menghitung
    return (this.#celsius * 9) / 5 + 32;
  }
}

const tm = new Termometer(30);
console.log(tm.celsius);          // 30   (tanpa tanda kurung)
console.log(tm.fahrenheit);       // 86
console.log(JSON.stringify(tm));  // "{}"  (field privat tidak ikut)
```

Listing 5: field privat

Ada dua tingkat perlindungan:

- `tm.#celsius` **di luar class** sudah ditolak saat file dibaca: `SyntaxError: Private field '#celsius' must be declared in an enclosing class`. Seluruh file tidak jalan.
- Getter tanpa setter berarti **hanya bisa dibaca**. `tm.fahrenheit = 1` melempar `TypeError: Cannot set property fahrenheit of #<Termometer> which has only a getter`.

2.4.2. Getter

Untuk apa: membuka data privat untuk dibaca, atau menyediakan nilai turunan (misalnya `fahrenheit` dari `celsius`), dengan cara pakai seperti field biasa.

Cara kerja: `get nama() { return ...; }` di dalam class. Dari luar dipanggil **tanpa tanda kurung**: `tm.celsius`, bukan `tm.celsius()`.

> **Catatan**
>
> Ada juga *setter* (`set celsius(v) { ... }`) yang dijalankan saat field di-assign. Setter berguna untuk validasi, misalnya menolak suhu di bawah −273. Di soal modul ini, perubahan saldo memakai method biasa (`setor`, `tarik`) karena method bisa mengembalikan pesan berhasil atau gagal.

2.5. extends dan super: Menurunkan Class

Untuk apa: membuat class yang **sama dengan** class lain **ditambah** sesuatu. Contohnya motor adalah kendaraan yang punya isi silinder. Semua yang dimiliki kendaraan otomatis dimiliki motor.

```
class Kendaraan {
  constructor(merk) {
    this.merk = merk;
  }
  info() {
    return `Kendaraan ${this.merk}`;
  }
  jalan() {
    return `${this.merk} berjalan`;
  }
}

class Motor extends Kendaraan {
  constructor(merk, cc) {
    super(merk);              // jalankan constructor Kendaraan dulu
    this.cc = cc;             // baru tambah field milik Motor
  }
  info() {                    // menimpa (override) info milik Kendaraan
    return `${super.info()} (${this.cc} cc)`;   // panggil versi induk
  }
}

const m = new Motor("Honda", 150);
console.log(m.info());                // "Kendaraan Honda (150 cc)"
console.log(m.jalan());               // "Honda berjalan"   (diwarisi, tidak ditulis ulang)
console.log(m instanceof Motor, m instanceof Kendaraan);   // true true
```

Listing 6: extends, super, dan menimpa method

Tiga aturan:

1. **`super(...)` di constructor dipanggil paling awal**, sebelum memakai `this`. Jika `this` dipakai lebih dulu, muncul `ReferenceError: Must call super constructor in derived class before accessing 'this' or returning from derived constructor`.
   Jika constructor turunan tidak memanggil `super` sama sekali, error yang sama muncul saat `new`.
2. **`super.namaMethod()` memanggil versi induk.** Ini dipakai saat menimpa method tetapi masih ingin memakai hasil induknya. Hasil induk tidak perlu disalin.
3. **Field privat induk tidak bisa diakses turunan.** `this.#saldo` di dalam class turunan ditolak dengan `SyntaxError` yang sama seperti di luar class. Turunan harus memakai getter dan method publik induknya.

Jalannya `new Motor("Honda", 150)`:

Langkah	Kode yang jalan	Isi object
1	constructor `Motor` mulai	belum boleh disentuh
2	`super("Honda")` : constructor `Kendaraan`	`{ merk: "Honda" }`
3	`this.cc = 150`	`{ merk: "Honda", cc: 150 }`

2.6. Class yang Menyimpan Object Lain

Untuk apa: mengelola banyak object sekaligus. Contohnya perpustakaan menyimpan banyak buku, atau bank menyimpan banyak rekening. Relasinya mirip tabel induk dan tabel anak di database.

Cara kerja: class pengelola punya field array. Method-nya memakai method array dari Modul 3: `push` untuk menambah, `find` untuk mencari lewat id (seperti mencari baris lewat primary key), dan `reduce` untuk menjumlah.

```
class Buku {
  constructor(kode, judul, stok) {
    this.kode = kode;
    this.judul = judul;
    this.stok = stok;
  }
}

class Perpustakaan {
  constructor() {
    this.koleksi = [];                                  // banyak Buku
  }
  tambah(buku) {
    this.koleksi.push(buku);
    return this.koleksi.length;
  }
  cari(kode) {
    return this.koleksi.find((b) => b.kode === kode);   // WHERE kode = ... LIMIT 1
  }
  totalStok() {
    return this.koleksi.reduce((acc, b) => acc + b.stok, 0);
  }
  pinjam(kode) {
    const b = this.cari(kode);                          // pakai method sendiri
    if (b === undefined) return `GAGAL: buku ${kode} tidak ada`;
    if (b.stok === 0) return `GAGAL: ${b.judul} habis`;
    b.stok = b.stok - 1;
    return `Pinjam ${b.judul}, sisa ${b.stok}`;
  }
}

const perpus = new Perpustakaan();
perpus.tambah(new Buku("B1", "Bumi", 2));
perpus.tambah(new Buku("B2", "Pulang", 0));
console.log(perpus.pinjam("B1"));   // "Pinjam Bumi, sisa 1"
console.log(perpus.pinjam("B2"));   // "GAGAL: Pulang habis"
console.log(perpus.pinjam("B9"));   // "GAGAL: buku B9 tidak ada"
console.log(perpus.totalStok());    // 1
```

Listing 7: perpustakaan menyimpan buku

Perhatikan bentuk `pinjam`. Polanya sama dengan corong di Modul 3: cek yang gagal di atas dengan `return` lebih awal, lalu kerjakan perubahan di bawah. Method `pinjam` juga memakai `cari`, method miliknya sendiri, lewat `this.cari(...)`. Satu method mengerjakan satu hal, lalu method lain memakainya.

> **⚠ Perhatian**
>
> `find` mengembalikan **object yang sama** yang ada di dalam array, bukan salinan (ingat referensi di Modul 4). Karena itu `b.stok = b.stok - 1` mengubah buku di dalam `koleksi`. Itulah yang diinginkan di sini. Jika object yang ditemukan punya field privat, ubah lewat method-nya, misalnya `rek.tarik(...)`.

2.7. Membaca UML Class Diagram

Soal modul ini memberi struktur class dalam UML class diagram. Satu kotak adalah satu class, dengan tiga bagian: nama, field, dan method.

Tanda	Arti
`+ nama : string`	field publik bertipe string
`- #saldo : number`	field privat
`+ setor(nominal : number) : string`	method dengan parameter bertipe number, mengembalikan string

garis dengan segitiga kosong	pewarisan: class di ujung tanpa segitiga `extends` class yang ditunjuk segitiga
garis dengan belah ketupat kosong	agregasi: class di sisi belah ketupat menyimpan banyak object class di ujung lain
`0..*`	nol atau banyak, sama dengan notasi kardinalitas di ERD

2.8. Membaca Pesan Error

Pesan error	Artinya	Penyebab umum
`Class constructor X cannot be invoked without 'new'`	Class dipanggil seperti fungsi biasa.	Lupa `new`.
`Cannot read properties of undefined (reading 'x')` di dalam method	`this` bernilai `undefined`.	Method dijadikan callback tanpa dibungkus arrow function.
`Private field '#x' must be declared in an enclosing class`	Field privat diakses di luar class pemiliknya. Ini `SyntaxError`, jadi seluruh file tidak jalan.	Akses dari luar class atau dari class turunan.
`Cannot set property x of #<X> which has only a getter`	Assign ke getter yang tidak punya setter.	Mencoba mengubah data yang sengaja dibuat hanya-baca.
`Must call super constructor in derived class before accessing 'this'`	Constructor turunan memakai `this` sebelum `super(...)`, atau tidak memanggil `super`.	Urutan baris di constructor.

✎ Catatan

Saat live coding, pertanyaan “object vs class” sering muncul. Jawaban yang baik: object literal cocok untuk data satu kali pakai. Class adalah cetakan untuk banyak object yang punya perilaku sama dan aturan yang harus dijaga (enkapsulasi dengan field privat). Di TypeScript nanti ada `interface`, yang hanya mendeskripsikan **bentuk** data tanpa perilaku dan hilang setelah di-compile.

Coba Sendiri

Kerjakan di fungsi `cobaSendiri()` pada file soal `Jawaban/soal-js5.js` (sudah tersedia di starter). Sebelum menjalankan, **tulis tebakanmu** sebagai komentar di ujung setiap baris `console.log`. Jalankan bagian ini saja dengan `node Jawaban/soal-js5.js coba`, lalu bandingkan.

```
class Kucing {
  constructor(nama) {
    this.nama = nama;
  }
  suara() {
    return `${this.nama}: meong`;
  }
}

const k = new Kucing("Oyen");
console.log(k.suara());
console.log(typeof Kucing, k instanceof Kucing);

const objek = { nama: "Belang", suara() { return `${this.nama}: meong`; } };
console.log(objek.suara());

class Singa extends Kucing {
```

```
  suara() {
    return super.suara().replace("meong", "auum");
  }
}
console.log(new Singa("Leo").suara());
console.log(JSON.stringify(k));

const lepas = k.suara;
```

1. Tebakan mana yang salah? Jelaskan penyebabnya.
2. `objek` dan `k` sama-sama punya `nama` dan `suara()`. Kapan kamu memilih class, kapan object literal?
3. Tambahkan `console.log(lepas());` di baris terakhir. Catat error-nya dan jelaskan isi `this` saat itu.
4. Tambahkan getter `namaBesar` di `Kucing` yang mengembalikan `nama` dalam huruf kapital, lalu cetak `k.namaBesar`. Mengapa tidak ditulis `k.namaBesar()`?

Jawab pertanyaan di atas di bawah baris `// JAWABAN Coba Sendiri N` pada file soal.